

Application Development 2 (Mobile)

Week 1 - Foundations & First Run

What stands between an idea and a phone?

You have an idea for an app. You have a phone in your pocket.

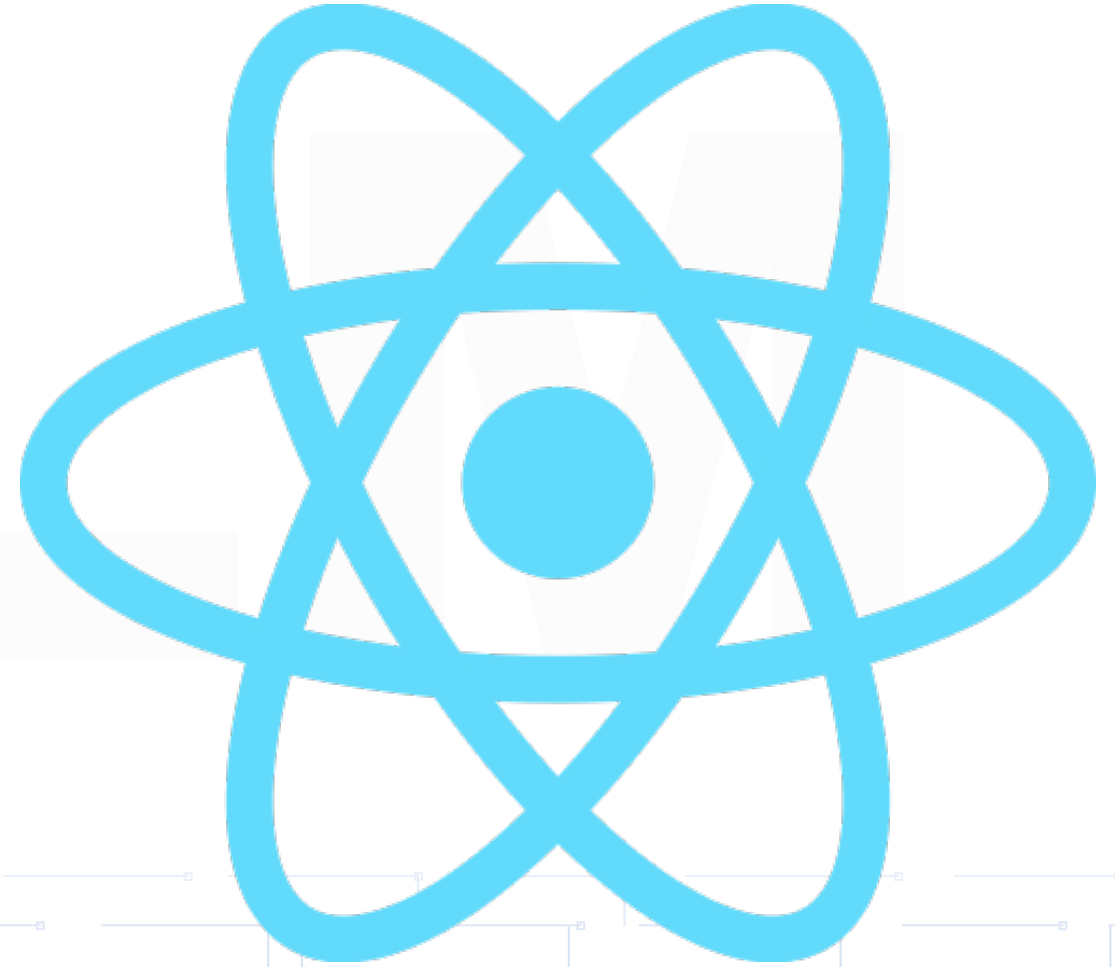
This week we close the gap between the two.

By the end of the week: a working toolbox, an app you wrote running in front of you, and that code safe on GitHub.



Why React Native

- One codebase in JavaScript/TypeScript -> a real native app on **both** Android and iOS.
- You write **components**; the framework renders real native UI (not a web page in a wrapper).
- The skills transfer: it is React, the same model used across the industry for web and mobile.



React Native in one idea

A screen is a function that returns UI:

```
function Welcome() {  
  return (  
    <View style={styles.screen}>  
      <Text style={styles.title}>Welcome to Wanderlist</Text>  
    </View>  
  );  
}
```

No HTML. View and Text are **native** building blocks. We compose small functions into whole screens.

Two worlds: your computer and the device

Every tool we install this week belongs to one of two places:

- **Your computer** - the workshop: the editor and the tools that build and serve your app.
- **The device** - an emulator or a phone: where the app actually lives and runs.

Mixing these two up is the #1 source of week-one confusion. So before installing anything, let's place every tool on its side of the line.

What is Node.js - and why is it in a mobile course?

- JavaScript was born inside the browser and, for years, could run nowhere else.
- **Node.js** lets JavaScript run **on your computer**, the way Python or Java programs do.
- The mobile tools we use - Expo's command line, the Metro bundler - are themselves **JavaScript programs**. They need Node.js to run.

Node.js never runs on the phone. It powers the workbench on your computer; the device runs the app.



npm and npx - Node's two helpers

Both install automatically with Node.js:

- **npm** - the package manager. Downloads the libraries your project lists in `package.json` (React Native itself arrives this way).
- **npx** - "run this tool now, without installing it for good". Fetches a command-line tool and executes it once.

```
npx create-expo-app@latest wanderlist    → npx runs Expo's project generator  
npm install                             → npm restores a project's libraries
```

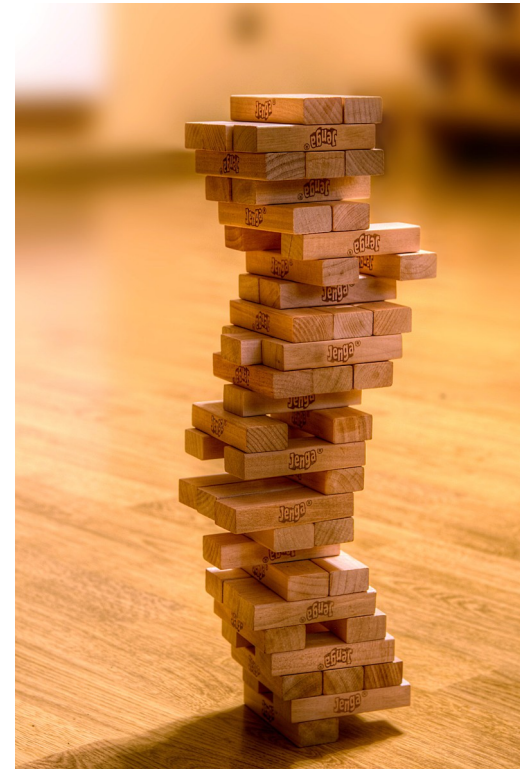
Install checklist - in this order

- 1 **Node.js** (the LTS version from nodejs.org, v20+)
- 2 **VS Code** (code.visualstudio.com)
- 3 **Git (git-scm.com) and a free account on github.com**
- 4 **Android Studio** (developer.android.com/studio)
- 5 **The Pixel 10 virtual device** (created inside Android Studio)

LEOM

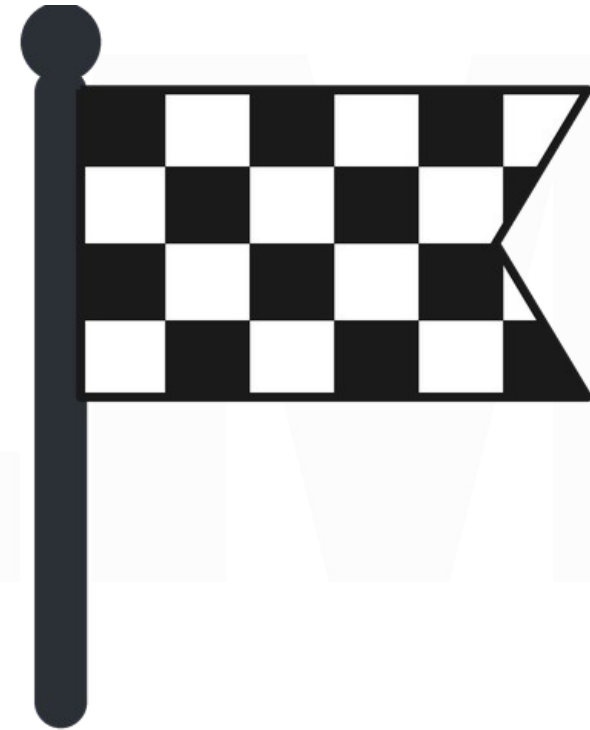
npm install

Added 4 packages.
And 1,400 you have never heard of, holding
hands.



Checkpoint - the installs start now

That is the map: which tool lives where, and why. Now the first three go in - Node.js, VS Code and Git - with a check after each one.



Lab 1 - Node.js and VS Code

20 minutes. Install Node.js (the LTS version from nodejs.org), then VS Code. Keep the installer defaults.

Then open a new terminal and ask Node for its version - the next slide shows how.

LEOM

Does the command line answer?

Open a fresh terminal (macOS: Terminal, Windows: PowerShell) and ask:

```
node --version    → v20 or higher  
npm --version     → any version is fine  
git --version     → any version is fine
```

"command not found"? The tool is not installed - or the terminal was already open during the install. Close it and open a new one.

Lab 2 - Git and your GitHub account

20 minutes. Install Git from git-scm.com with the installer defaults, then create a free account on github.com - pick a username you are happy to show an employer.



How your code reaches the screen

you edit `src/app/index.tsx` and hit SAVE

|

v

Metro bundler - a Node.js program on YOUR COMPUTER

|

packs all your JavaScript into one bundle

v

Expo Go - on the EMULATOR or a phone

downloads the bundle and runs it

- This is why the terminal running `npx expo start` must **stay open** - it is feeding your app.
- This is why a phone must share your computer's **Wi-Fi** - it downloads the bundle from it.

One standard setup - our source of truth

- Everyone runs the app on the **Android Studio emulator**, device profile **Pixel 10**.
- Same screen, same behaviour, same bugs -> when something breaks, we all debug the **same** machine.
- Help in class, demos, and grading are all judged on this target.
- **Optional extras**, never required: Expo Go on your own phone; the iOS simulator (Mac only).

LEOM

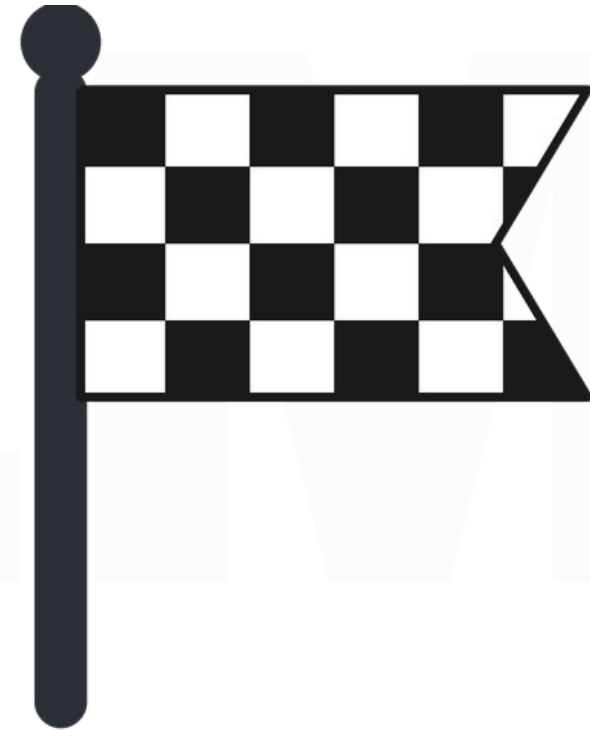
Have you tried

turning it off and on again?

Reload the app. Restart the bundler. Then start debugging.

Checkpoint - the emulator build starts now

You know where the app runs and how your code gets there. Now we build that path for real: Android Studio, the Pixel 10, and a first app running on it.



Lab 3 - Android Studio and the Pixel 10

45 minutes. Install Android Studio, let it download the Android SDK, then create and boot the Pixel 10 - the next three slides walk you through it.



Install Android Studio

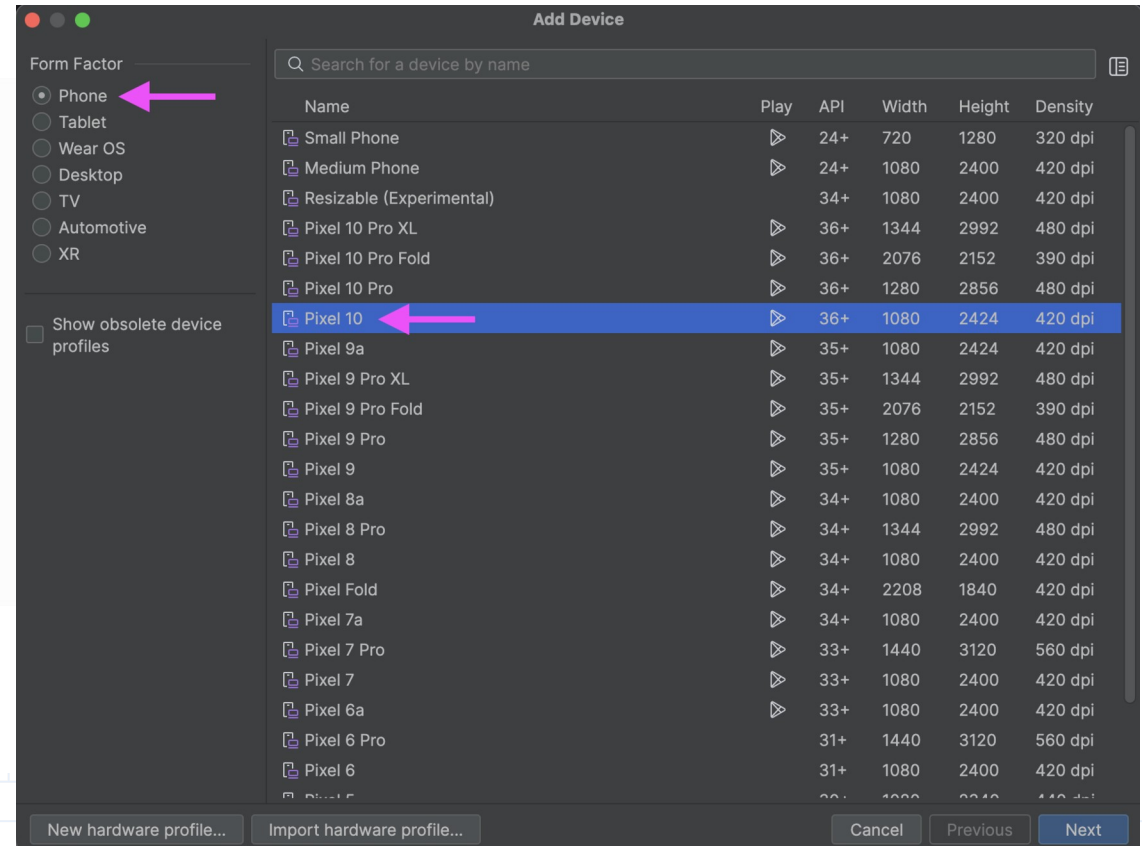
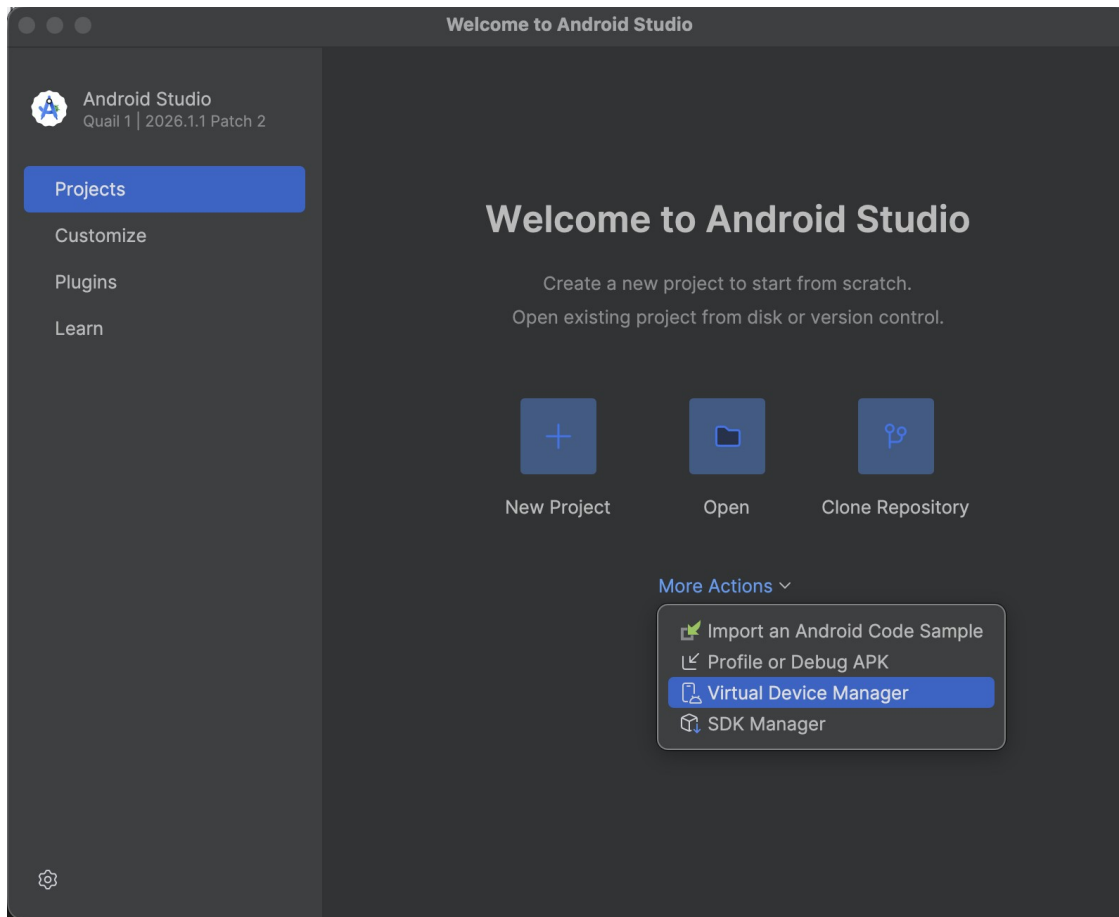
- 1 **Download from developer.android.com/studio and run the installer** (keep the defaults)
- 2 **On first launch the setup wizard downloads the Android SDK - accept the standard setup**
- 3 **Wait it out** (the SDK is several GB - plan for ~15 GB of free disk and some patience)

Create the Pixel 10 emulator

- 1 In Android Studio open the **Device Manager** (from the welcome screen or Tools menu)
- 2 Create Virtual Device -> category Phone -> pick **Pixel 10**
- 3 **System image: take the newest one marked Recommended** (download it if asked)
- 4 Finish, then press the play button beside **Pixel 10**

Checkpoint: an Android home screen boots on your monitor. The first boot is slow - later ones are fast.

Create the Pixel 10 emulator - where to click



Create your first app

```
npx create-expo-app@latest wanderlist  
cd wanderlist  
npx expo start
```

- create-expo-app generates the project folder; expo start fires up **Metro** and prints a QR code.
- Keep this terminal open - it is the bundler feeding your app.



Run it on the emulator

- 1 **Boot the Pixel 10 from the Device Manager** (skip if it is already running)
- 2 **In the terminal where `npx expo start` is running, press `a`**
- 3 **Expo installs Expo Go into the emulator and opens your app inside it**

Checkpoint: the starter screen shows in the emulator. Edit code, save, and it **reloads instantly**.

Lab 4 - your app on your Pixel 10

35 minutes. Create wanderlist with the three commands from two slides back, then press **a** with the Pixel 10 booted. Keep that terminal open.



Optional: your own phone (or iOS)

- Install the **Expo Go** app on your phone (App Store / Play Store).
- Phone and computer on the **same Wi-Fi**, then scan the QR code from the terminal.
- On a Mac with Xcode, pressing **i** opens the iOS simulator instead.

ALM

Anatomy of the project

```
wanderlist/  
  src/  
    app/      screens (file-based routing)  
      index.tsx the first screen you see  
  assets/     images, fonts  
  package.json dependencies + scripts
```

The screen at `src/app/index.tsx` is what loads first. That is the file we edit this week.

A quick JavaScript refresher

```
const title = "Wanderlist";           // const / let, not var
const greet = (name) => `Hi ${name}`; // arrow fn + template literal
const { name, notes } = place;        // destructuring
import { View } from "react-native"; // ES modules
export default Index;
```

You know this from earlier blocks - we will lean on it all term.

JSX - describing UI as code

```
<View style={styles.screen}>  
  <Text style={styles.title}>Welcome to Wanderlist</Text>  
  <Text>Your next trip starts here.</Text>  
</View>
```

- Looks like HTML, but it is JavaScript.
- { } drops a JavaScript value into the UI: <Text>{userName}</Text>.

Your first component

```
import { View, Text, StyleSheet } from "react-native";

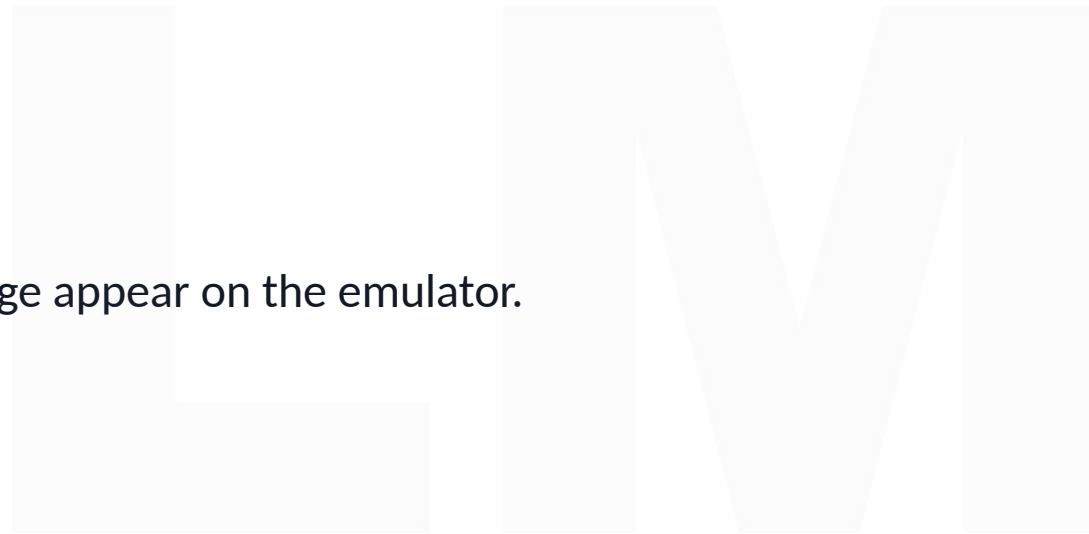
export default function Index() {
  const appName = "Wanderlist";
  return (
    <View style={styles.screen}>
      <Text style={styles.title}>Welcome to {appName}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  screen: { flex: 1, alignItems: "center", justifyContent: "center" },
  title: { fontSize: 24, fontWeight: "600" },
});
```

Core native components

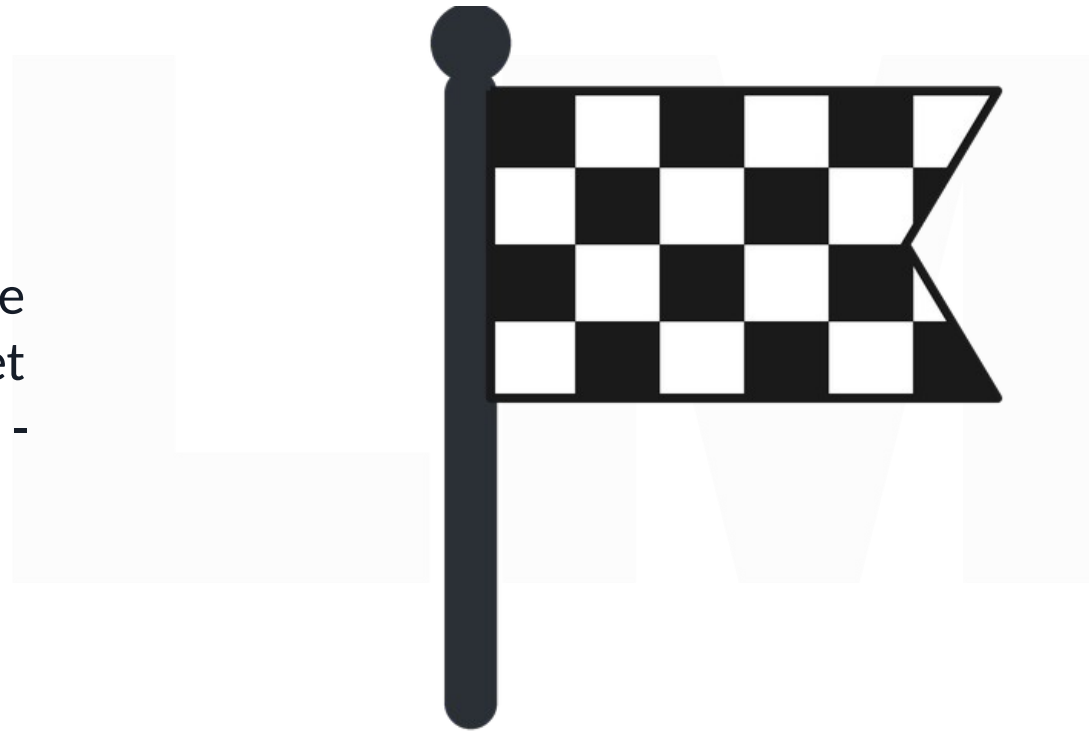
- View - a container (think div).
- Text - all text must live inside a Text.
- Image - pictures.
- ScrollView - content that scrolls.

Save the file with Expo running and watch the change appear on the emulator.



Checkpoint - live build starts now

JSX and components, on a slide, are half the picture. Now watch the welcome screen get built from the starter file, live, in the editor - then build yours.



Lab 5 - your welcome screen

20 minutes. Replace everything in `src/app/index.tsx`: a View that centers its content, and a Text that says "Welcome to Wanderlist". Keep the app name in a const and render it with `{ }` - never type it twice.



Git and GitHub - where your work lives

Your work lives only on your laptop. One disk failure erases the course.

- **Git** - a tool on your computer that keeps named snapshots of your project (commits) you can always return to.
- **GitHub** - a website that keeps a copy of those snapshots online.



Your first commit

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"  
git init  
git add .  
git commit -m "Week 1: welcome screen"
```

- The two config lines: once per computer, with the email of your GitHub account.
- `git init`: once per project.
- `git add` and `git commit`: every time you want a snapshot.

Push it to GitHub

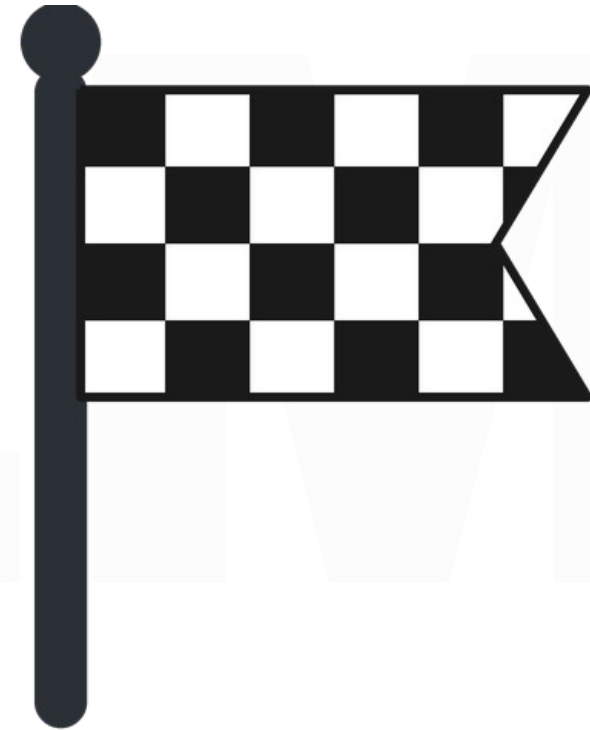
Create an empty repository named wanderlist on github.com first - no README, no licence. Then:

```
git remote add origin https://github.com/<you>/wanderlist.git  
git branch -M main  
git push -u origin main
```

Your work now lives off your laptop. From this week on, **every week ends with a push.**

Checkpoint - your first push, live

Snapshot on your computer, copy on GitHub.
Watch one push go through, then send yours -
the week ends when your repository shows the
welcome screen's code.



Lab 6 - your first push

35 minutes. In a second terminal inside wanderlist: the two git config lines, then init, add, commit. Create an empty wanderlist repository on github.com, connect it, push.

LM

Your Week 1 hand-in: a short reflection

No code to submit this week. Instead: half a page, via the Lea Submission, in your own words:

- What does **Node.js** do in our setup - and where does each tool run: your computer, or the device?
- What happens between saving a file in VS Code and seeing the change on the emulator?
- Where are you right now: does the app run on your Pixel 10, and is it on GitHub (paste the link)? Where did setup snag - or where are you still stuck?
- What would **you** like to build by the end of this course?

This is where Wanderlist begins

Today's screen is tiny on purpose. Over the next weeks the **same app** grows a feature at a time - cards, a form, a list, navigation, live data, a photo - into something you would show an employer.



When something breaks

- Pressing a does nothing -> is the Pixel 10 booted in the Device Manager first?
- Emulator is up but the app never opens -> stop (Ctrl+C) and `npx expo start -c` to clear the cache.
- `node: command not found` -> Node.js is missing, or reopen the terminal after installing.
- `Author identity unknown on commit` -> run the two `git config` lines, then commit again.
- `git push` refuses your password -> GitHub wants a browser sign-in or a personal access token, not your account password.
- QR will not scan (phone route) -> phone and computer on the same Wi-Fi? VPN enabled?

One kick, ten thousand times

I fear not the person who practised 10,000 kicks once,
but the one who practised one kick 10,000 times.

